

Homework Notes

1. Overview

- Homework you will be simulating Thread Race 2000.
 - Is about communication between threads and locks.
 - We simulate a race between a collection of threads.

2. Thread Race 2000

- Program will take on CLI input.
 - must be an int
 - strictly larger than 0.
 - The input is the number of threads that will compete in the race.
 - if wrong value given produce an error.
 - Ex `./bin/race 3`
- r = # number of racers.
- Your program will spawn $r + 1$ threads from main.
- Once they are all completed, program will output the winners.
 - One thread will be game master who runs the race.
 - The other threads will be the racers.

Game Master Thread

- generates random dice rolls.

Basic Logic

While all threads have not crossed the finish line

 Select a random number k between 0 and 5 (inclusive)

Roll a k 6-sided dice (random number 1-6 inclusive)
Add the dice rolls to queue
Sleep for a random length between 0-2 seconds.

Racers

- Racers will then use these dice rolls to move across the race track.
- A racer wins when it has move 20 or more spaces.
- Once the racer is at the end of the track it has crossed the finish line.
- Once ALL racers have crossed the finish line, the program is mover.

Basic Logic

While thread has not moved at least 20 spaces
 Read a dice roll from the queue
 Move forward that many spaces
 Sleep for a random length between 0-2 seconds.

Main Program

- Should print a list of threads ranked by when they crossed the finish line.
- All threads will sleep during the race to simulate the program doing a complex calculation.

Allowed To Use

- Can use semaphores, locks, and/or conditional variables to manage the threads.
 - Can use semaphore class proved in lecture as part of solution.
 - Must protect all the shared data structures.
- You CANNOT use the thread safe queue class from lecture.

- It is possible for one thread to grab multiple dice rolls before another thread executed based on how long it sleeps.
 - This is allowed as long as each thread sleeps for a random time between grabbing another dice roll from the queue, it may keep running forward. It is a race
- Can use basic rand command or use C++11 non-deterministic hardware entropy based random number generator.

Must Use

- Must use the queue from the std library to pass the dice rolls between threads.
 - Do not build your own.
- You must make sure the following features are thread safe:
 - I/O two threads should never write to cout at the same time
 - Dice Queue must be safe for the game master and racer
- You must make sure to avoid the following:
 - Deadlock: No thread can race forward (the program stops forever because no one can move)
- Starvation: all threads must finish the race (don't let one thread never get a dice roll)
 - Once you have completed your code, you will run a few tests and write your thoughts on how your code works.
 - In README

Example

```

1 ./bin/race 3
2 Starting Race with 3 threads.
3 Thread 140628519687936 has left the gate.
4 Thread 140628519687936 moved forward 2 spaces.
5 Thread 140628519687936 moved forward 6 spaces.
6 Thread 140628519687936 moved forward 1 spaces.
7 Thread 140628511295232 has left the gate.
8 Thread 140628502902528 has left the gate.
9 Thread 140628502902528 moved forward 5 spaces.
10 Thread 140628519687936 moved forward 4 spaces.
11 Thread 140628519687936 moved forward 4 spaces.
12 Thread 140628511295232 moved forward 5 spaces.
13 Thread 140628519687936 moved forward 2 spaces.
14 Thread 140628502902528 moved forward 4 spaces.
15 Thread 140628519687936 moved forward 1 spaces.
16 Thread 140628519687936 has crossed the finish line.
17 Thread 140628511295232 moved forward 3 spaces.
18 Thread 140628511295232 moved forward 3 spaces.
19 Thread 140628511295232 moved forward 3 spaces.
20 Thread 140628502902528 moved forward 2 spaces.
21 Thread 140628511295232 moved forward 1 spaces.
22 Thread 140628511295232 moved forward 2 spaces.
23 Thread 140628511295232 moved forward 5 spaces.
24 Thread 140628511295232 has crossed the finish line.
25 Thread 140628502902528 moved forward 1 spaces.
26 Thread 140628502902528 moved forward 6 spaces.
27 Thread 140628502902528 moved forward 4 spaces.
28 Thread 140628502902528 has crossed the finish line.
29 1: 140628519687936
30 2: 140628511295232
31 3: 140628502902528

```

README

1. Your name and drexel ID (abc123@drexel.edu)
2. Instructions to run you code.
3. Short Essay Question 1: What semaphores/condition variables did you use and why? (If you didn't use any, why not?)
4. Short Essay Question 2: What locks did you use and why? (if you didn't use any, why not?)
5. Short Essay Question 3: Why are you confident the program can never deadlock? (Explain how you

programmed it to avoid this)

6. Short Essay Question 4: Why are you confident the program can never let any thread starve? (Explain how you programmed it to avoid this)
7. Short Essay Question 5: What was the most challenging part of this assignment?

Makefile

- make - Builds the Program
- make run - Runs three races with 2, 4, and 10 racers.
- make clean - Remove compiled code
- make doc - generate doxygen documentation